TP 4: Algorithmes parallèles C++

Cours de Parallélisme

8 octobre 2026

Le but de ce TP est d'utiliser les algorithmes standards du C++ afin de les exécuter en parallèle. Il doit être réalisé sans utiliser de boucles `for`, `while` ou `do-while`.

Compilation : les algorithmes parallèles nécessitent au moins C++17 (nous utiliserons C++20) et, avec GCC, la bibliothèque TBB. Sans `-ltbb`, le code peut ne pas compiler ou s'exécuter séquentiellement, sans erreur.

```
g++ -std=c++20 -O3 main.cpp -o main -ltbb
```

Tâche 0

Demandez la création d'un compte sur le cluster de l'université en suivant les instructions données dans le fichier « HPC access » sur Moodle.

Tâche 1

Initialisez un vecteur d'entiers allant de 1 à 100 000 000 (400 Mo) et mélangez-le avec un générateur `std::mt19937` initialisé avec la graine `42`.

Triez-le séquentiellement, puis recréez le même vecteur mélangé (avec la même graine) et triez-le en parallèles. Mesurez les temps d'exécution avec le code suivant :

```cpp
const auto start = std::chrono::steady_clock::now();
// ...
const auto end = std::chrono::steady_clock::now();
std::cout << "Elapsed time: "
          << std::chrono::duration_cast<std::chrono::milliseconds>(end - start).count()
          << " ms" << std::endl;
```

Calculez le speedup $S = T_{seq}/T_{par}$ apporté par le parallélisme.

Tâche 2

Dans le code suivant, le message a été encodé en remplaçant chaque caractère par son indice dans l'alphabet donné. Effectuez la transformation inverse pour reconstruire le message original dans une `std::string`, puis affichez-le.

```cpp
#include <cstdint>
#include <string>
#include <vector>

const std::string alphabet = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!?,. ";
const std::vector<uint8_t> msg_keys = {
    48, 4, 11, 2, 14, 12, 4, 56, 8, 13,
    56, 15, 0, 17, 0, 11, 11, 4, 11, 8,
    18, 12, 56, 2, 11, 0, 18, 18, 56, 52
```

```
};
```

Tâche 3

Calculez séquentiellement la somme du vecteur `data` initialisé comme suit :

```
std::vector<float> data(1'000'000, 1.f);
data[0] = 1.0e8f;
data.back() = -1.0e8f;
```

Calculez en parallèle la somme de ce même vecteur 10 fois. Expliquez le comportement observé.

Note : l'effet observé n'apparaît qu'avec au moins 2 cœurs.

Remplacez le code initial par le suivant :

```
std::vector<float> data(1'000'000, 1.f);
data[0] = 1.0e8f;
data[1] = -1.0e8f;
```

Expliquez le changement observé.

Tâche 4

Initialisez un vecteur de float allant de 1.0 à $N = 1\,000\,000$.

Appliquez la fonction suivante à chaque élément du vecteur, puis calculez leur somme. Cette tâche est à réaliser de manière séquentielle :

```
#include <cmath>

float transformer(float x) {
    const float ratio = 0.1f;
    return std::sqrt(x) * ratio;
}
```

Implémentez cette transformation comme une lambda puis comme un foncteur.

Dans les trois cas (fonction libre, lambda, foncteur), comment faire pour que le ratio ne soit plus codé en dur, mais fourni par le code appelant ? Est-ce possible dans les trois cas ? Expliquez les avantages et inconvénients de chaque approche.

Note : le type de la valeur initiale de la somme (par exemple `0.f` ou `0.0`) détermine le type de l'accumulateur, donc la précision du résultat.

Tâche 5

Voici un tenseur 3D de dimensions $x \times y \times z = 3 \times 2 \times 4$:

```
constexpr size_t dim_x = 3;
constexpr size_t dim_y = 2;
constexpr size_t dim_z = 4;
// Indexed as tensor[z][y][x]
const std::vector<std::vector<std::vector<unsigned>>> tensor {
    { {1, 2, 3}, {3, 2, 1} },
    { {2, 3, 4}, {4, 3, 2} },
    { {6, 7, 8}, {8, 7, 6} },
    { {7, 8, 9}, {9, 8, 7} },
};
```

Ce stockage n'est pas optimal pour la mise en cache des données. Copiez les données dans un vecteur 1D tel que `data[z * dim_y * dim_x + y * dim_x + x] == tensor[z][y][x]`.

Note : Cette tâche peut être réalisée de différentes façons. Appeler un algorithme parallèle depuis un autre est autorisé, mais le standard ne garantit pas que l'appel interne s'exécute en parallèle : un seul niveau de parallélisme suffit généralement.

Tâche 6

Voici un tableau 2D irrégulier (dont les lignes ont des tailles différentes) :

```
const std::vector<std::vector<unsigned>> jagged {
    { 0, 0 },
    { 1 },
    { 2, 2, 2, 2 },
    { 3, 3, 3 },
};
```

Comme dans la tâche précédente, ce stockage ne permet pas une mise en cache optimale et empêche de paralléliser facilement une opération sur l'ensemble des données. Copiez ces données dans un vecteur 1D, ligne après ligne, en conservant l'ordre.

Tâche 7

Lors du TP précédent, vous avez implémenté le rendu de la fractale de Mandelbrot en utilisant OpenMP. Changez l'implémentation pour utiliser des algorithmes parallèles et comparez les temps d'exécution.

Utilisez les paramètres suivants (largeur, hauteur, nombre d'itérations maximal, x_{min}, x_{max}, y_{min}, y_{max}, nombre de threads, fichier de sortie), où 0 thread correspond au nombre par défaut :

```
./mandelbrot 4000 2000 300 -2.2 1.0 -1.5 1.5 0 mandelbrot.bmp
```

Note : les deux implémentations devraient avoir des temps d'exécution similaires. Si ce n'est pas le cas, cherchez à comprendre pourquoi.

Tâche bonus

Soit le vecteur de tuples {key; value} initialisé comme suit :

```
#include <random>
#include <tuple>
#include <vector>

const size_t N = 100'000;
const size_t key_max = N / 4;
std::vector<std::tuple<size_t, float>> data(N, { 0, 0.f });

// Random generator
std::random_device rd;
std::mt19937 engine(rd());
std::uniform_real_distribution<float> dist_f(0.f, 1.f);
std::uniform_int_distribution<size_t> dist_u(0, key_max - 1);

for(size_t i = 0; i < N; ++i) {
    data[i] = { dist_u(engine), dist_f(engine) };
}
```

Plusieurs tuples possèdent la même clé. Calculez en parallèle la somme des valeurs associées à la même clé, dans un `std::vector<float>` de taille `key_max` indexé par la clé. Pour cette tâche, vous pouvez utiliser les boucles traditionnelles dans une boucle parallélisée.

Attention : deux threads qui traitent des tuples de même clé ne doivent pas modifier la même somme en même temps. Comparez votre résultat à une somme séquentielle avec une tolérance, et non avec == : l'addition de flottants n'est pas associative.